

PROJETO DE NAVEGAÇÃO E INTERAÇÃO

Prof. André Mussato

Apostila 3 – Cadastrar e Listar Categoria

Projeto Pizzaria (parte 3):

Passo 1 — Criar middleware de autorização para administrador

Agora, dentro da pasta middlewares, crie um novo arquivo chamado isAdmin.ts.

Esse middleware será responsável por verificar se o usuário autenticado possui permissão de administrador antes de acessar determinadas rotas da aplicação.

Adicione o seguinte código:

```
TS isAdmin.ts X
src > middlewares > TS isAdmin.ts > ...
1  import { Request, Response, NextFunction } from "express";
2  // Importa os tipos do Express usados no middleware
3  // Request representa a requisição recebida
4  // Response representa a resposta enviada ao cliente
5  // NextFunction representa a função que permite seguir para o próximo middleware/controller
6
7  import prismaClient from "../prisma";
8  // Importa a instância do Prisma Client
9  // Ela permite consultar o banco de dados
10
11 export const isAdmin = async (
12   req: Request,
13   res: Response,
14   next: NextFunction
15 ): Promise<void> => {
16   // Cria e exporta um middleware chamado isAdmin
17   // Ele verifica se o usuário logado possui permissão de administrador
18   // O async permite usar await dentro da função
19   // Promise<void> indica que essa função não retorna dados diretamente
20
21   const user_id = req.user_id;
22   // Pega o ID do usuário salvo dentro da requisição
23   // Esse user_id normalmente foi adicionado antes pelo middleware isAuthenticated
24
25   if (!user_id) {
26     // Verifica se não existe user_id na requisição
27     // Isso indica que o usuário não está autenticado corretamente
28   }
```

```

28
29     res.status(401).json({
30         error: "Usuário sem Permissão",
31     });
32     // Retorna status 401, indicando que o usuário não tem autorização
33
34     return;
35     // Encerra a execução do middleware
36     // Isso impede que o código continue
37 }
38
39 const user = await prismaClient.user.findFirst({
40     // Busca no banco de dados um usuário com o ID recebido
41
42     where: {
43         id: user_id
44     }
45     // Define a condição da busca
46     // Procura o usuário cujo id seja igual ao user_id
47 });
48
49 if (!user) {
50     // Verifica se nenhum usuário foi encontrado no banco
51
52     res.status(401).json({

```

```

53         error: "Usuário sem permissão",
54     });
55     // Retorna erro 401, pois o usuário não existe ou não é válido
56
57     return;
58     // Para a execução do middleware
59 }
60
61 if (user.role !== "ADMIN") {
62     // Verifica se o tipo/perfil do usuário é diferente de ADMIN
63     // Se não for ADMIN, ele não pode acessar a rota protegida
64
65     res.status(401).json({
66         error: "Usuário sem permissão",
67     });
68     // Retorna erro 401 informando falta de permissão
69
70     return;
71     // Encerra a execução para não liberar acesso
72 }
73
74 // Usuário é admin... então segue normal...
75 next();
76 // Chama o próximo middleware ou controller da rota
77 // Só chega aqui se o usuário existir e tiver role ADMIN
78 }
79
80 //Resumo do fluxo:
81 //Pega user_id -> busca usuário no banco -> verifica se existe -> verifica se é ADMIN -> libera acesso

```

Para que serve esse comando?

Esse middleware é utilizado para proteger rotas que só podem ser acessadas por administradores.

Ele realiza as seguintes verificações:

- verifica se o usuário está autenticado
- busca o usuário no banco de dados
- valida se o perfil (role) é ADMIN
- libera acesso apenas para administradores

Passo 2 — Criar schema categoria

Agora, dentro da pasta schemas, crie um novo arquivo chamado categorySchema.ts.

Esse schema será responsável por validar os dados enviados no cadastro de categorias.

Adicione o seguinte código:

```
src > schemas > TS categorySchema.ts > ...
1  // Importa a biblioteca Zod
2  // O Zod é usado para validação de dados e criação de schemas
3  // Ele ajuda a garantir que os dados recebidos estejam corretos
4  import { z } from 'zod';
5
6
7  // Cria e exporta um schema chamado createCategorySchema
8  // Esse schema será usado para validar os dados enviados na requisição
9  export const createCategorySchema = z.object({
10
11
12      // Define que será validado o corpo da requisição (req.body)
13      body: z.object({
14
15          name: z
16              // Define que o campo "name" obrigatoriamente deve ser do tipo string
17              // Caso não seja texto, retorna essa mensagem personalizada
18              .string({ message: "Categoria precisa ser um texto" })
19
20              .min(2, {
21                  // Define que o texto precisa ter no mínimo 2 caracteres
22                  // Caso tenha menos, retorna essa mensagem de erro personalizada
23                  message: "O nome da categoria precisa ter no mínimo 2 letras"
24              }),
25      });
26  });
```

Para que serve esse comando?

Esse schema é responsável por validar os dados enviados no cadastro de categorias.

Ele garante que:

- o campo name exista
- o valor seja um texto (string)
- o nome tenha pelo menos 2 caracteres

Passo 3 — Criar o service de categoria

Agora, dentro da pasta **services**, crie uma nova pasta chamada **category** e, dentro dela, o arquivo **CreateCategoryService.ts**.

Esse service será responsável pela lógica de criação de categorias no banco de dados.

Adicione o seguinte código:

```

34         name: name
35     },
36
37     // Define quais campos serão retornados após a criação
38     // Isso evita retornar informações desnecessárias
39     select: {
40
41         // Retorna o ID da categoria criada
42         id: true,
43
44         // Retorna o nome da categoria
45         name: true,
46
47         // Retorna a data de criação da categoria
48         createdAt: true,
49     }
50 })
51
52 // Retorna a categoria criada
53 return category;
54
55 // Captura possíveis erros durante a criação
56 } catch (erro) {
57
58     // Lança um novo erro personalizado
59     // Esse erro pode ser tratado no middleware global de erros
60     throw new Error("Falha ao criar Categoria")
61 }
62 }
63 }
64
65 // Exporta o service para ser utilizado em controllers ou outros arquivos
66 export { CreateCategoryService }

```

Para que serve esse comando?

Esse service é responsável por criar novas categorias no banco de dados. Ele:

- recebe os dados da categoria
- realiza o cadastro utilizando o Prisma
- retorna apenas os campos necessários (select)
- trata possíveis erros durante a operação

Passo 4 — Criar o controller de categoria

Agora, dentro da pasta **controllers**, crie uma nova pasta chamada **category** e, dentro dela, o arquivo **CreateCategoryController.ts**.

Esse controller será responsável por receber a requisição de criação de categorias e retornar a resposta para o cliente.

Adicione o seguinte código:

```

src > controllers > category > TS CreateCategoryController.ts > ...
13 // Controllers recebem os dados da requisição e retornam respostas
14 class CreateCategoryController {
15
16     // Método assíncrono responsável por processar a requisição
17     // O async permite usar await dentro do método
18     async handle(req: Request, res: Response) {
19
20         // Pega o campo "name" enviado no corpo da requisição
21         // Exemplo:
22         // {
23         //   "name": "Bebidas"
24         // }
25         const { name } = req.body;
26
27         // Cria uma instância do service de criação de categoria
28         const createCategory = new CreateCategoryService();
29
30         // Chama o método execute do service
31         // Envia o nome da categoria para ser salvo no banco
32         // O await espera a operação terminar
33         const category = await createCategory.execute({ name: name });
34
35         // Retorna status HTTP 201 (Created)
36         // Indica que um novo registro foi criado com sucesso
37         // Também retorna os dados da categoria criada em JSON
38         res.status(201).json(category)
39     }
40 }
41
42
43 // Exporta o controller para ser utilizado nas rotas da aplicação
44 export { CreateCategoryController }

```

Para que serve esse código?

Esse controller é responsável por intermediar a criação de categorias na aplicação.

Ele:

- recebe os dados enviados na requisição
- chama o service responsável pela lógica de negócio
- retorna a resposta HTTP para o cliente

Passo 5 — Adicionar a rota

Agora, no arquivo routes.ts, adicione os códigos:

```
58
59 router.post(
60   "/category", // Endpoint da rota
61   isAuthenticated, // Verifica se o usuário está autenticado
62   isAdmin, // Verifica se o usuário possui permissão de administrador
63   validateSchema(createCategorySchema), // Valida os dados enviados na requisição
64   new CreateCategoryController().handle // Executa o controller responsável por criar a categoria
65 );
66
```

Para que serve esse código?

Essa rota permite cadastrar novas categorias no sistema.

Ela funciona em etapas:

1. Verifica se o usuário está autenticado
2. Verifica se o usuário possui perfil ADMIN
3. Valida os dados enviados utilizando o schema
4. Executa o controller responsável pelo cadastro
5. Salva a categoria no banco de dados

Passo 6 — Testar cadastro de categoria

Agora, com o projeto em execução, teste o cadastro de uma nova categoria pelo Postman.

Para isso, é necessário estar logado e enviar o token do usuário no header da requisição.

Configuração no Postman

- **Método:** POST
- **URL:** http://localhost:3333/category
- **Authorization:** Bearer Token
- **Body (JSON):**

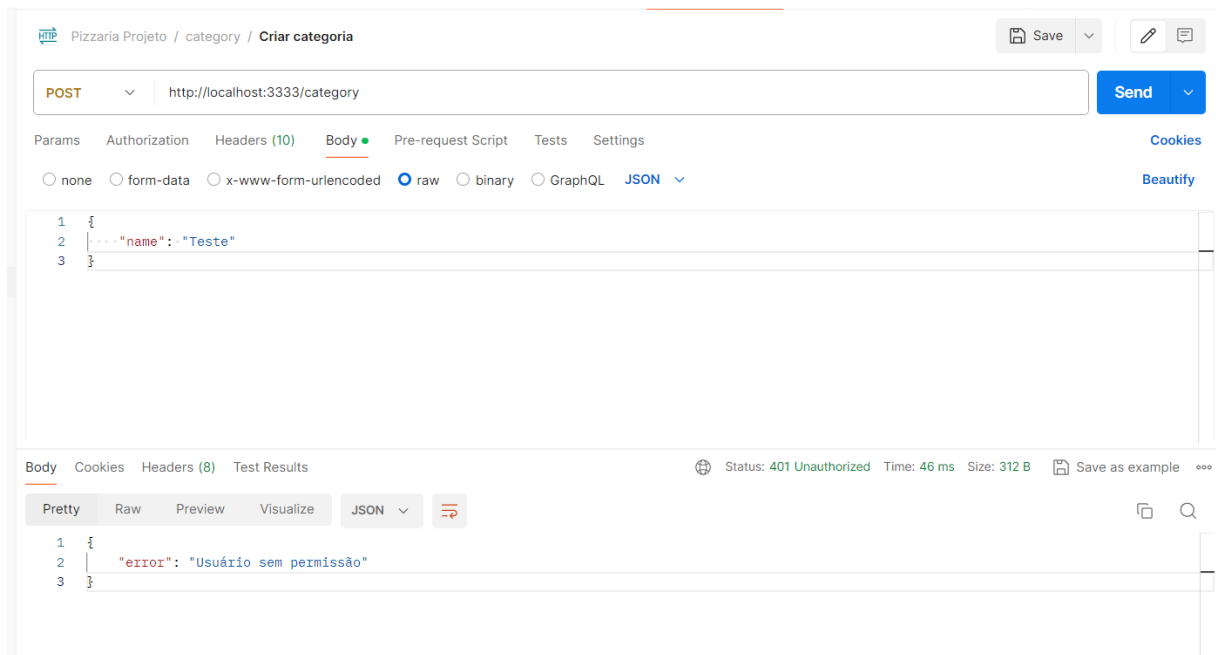
```
{
  "name": "Bebidas"
}
```

O que pode acontecer?

Ao tentar cadastrar a categoria, é possível que apareça a mensagem:

```
{
  "error": "Usuário sem permissão"
}
```

Isso acontece porque a rota está protegida pelo middleware isAdmin, ou seja, apenas usuários com perfil ADMIN podem cadastrar categorias.



Para que serve esse teste?

Esse teste verifica se a proteção da rota está funcionando corretamente.

Ele confirma que:

- o token está sendo validado
- a rota exige autenticação
- apenas usuários administradores podem cadastrar categorias
- usuários comuns não conseguem acessar funcionalidades restritas

Esse comportamento mostra que a autorização por perfil está funcionando corretamente.

Passo 7 — Alterar o perfil de um usuário para ADMIN

Agora, dentro do **pgAdmin**, escolha um usuário cadastrado e altere seu perfil (role) para ADMIN.

Execute o seguinte comando SQL:

UPDATE users

SET role = 'ADMIN'

WHERE id = 'id de usuário existente no banco';

O que esse comando faz?

- UPDATE users → altera dados da tabela users
- SET role = 'ADMIN' → define o perfil do usuário como administrador
- WHERE id = '...' → altera apenas o usuário com o ID informado



Para que serve essa alteração?

Essa alteração concede permissões administrativas ao usuário selecionado. Com isso, o usuário poderá:

- cadastrar categorias
- acessar rotas protegidas por isAdmin
- utilizar funcionalidades restritas do sistema

Esse teste é importante para validar o controle de permissões da aplicação.

Passo 8 — Teste cadastro de categoria novamente

Agora, com o usuário configurado como ADMIN, realize novamente o teste de cadastro de categoria no Postman.

É necessário:

- estar autenticado
- enviar o token JWT no header da requisição
- utilizar o token do usuário que possui perfil ADMIN

Caso o login tenha sido realizado com outro usuário, será necessário atualizar o token no Postman.

Configuração no Postman

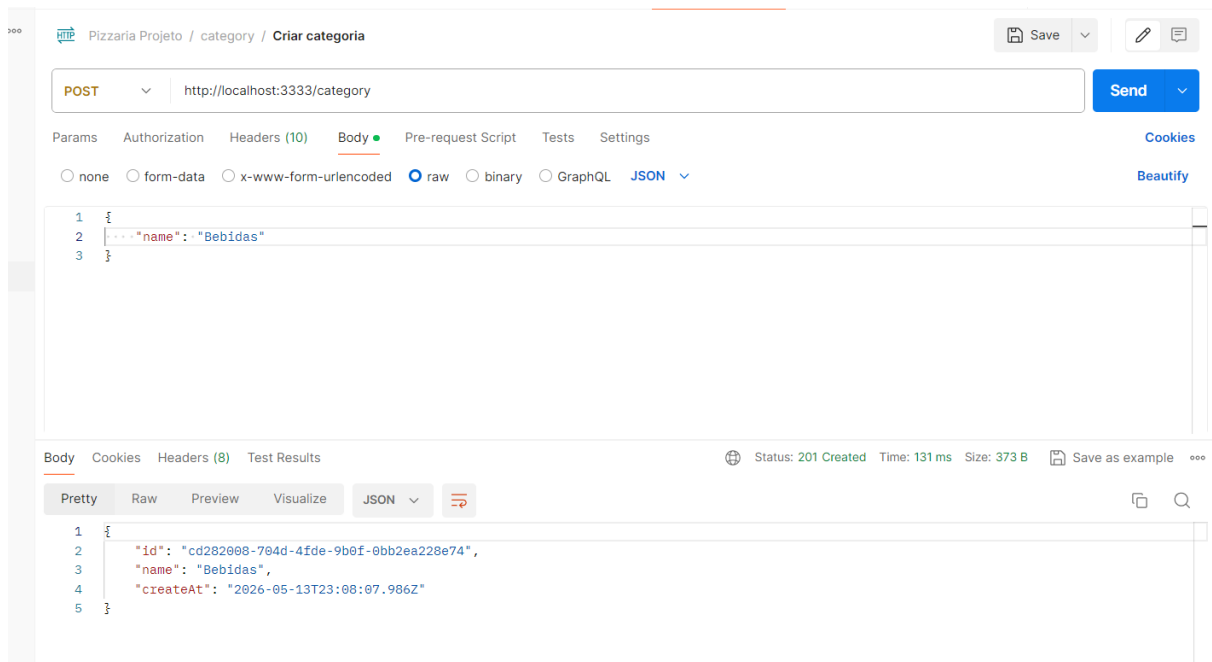
- **Método:** POST
- **URL:** http://localhost:3333/category
- **Authorization:** Bearer Token

Body (JSON)

```
{
  "name": "Bebidas"
}
```

The screenshot shows the Postman interface for a POST request. The URL is set to http://localhost:3333/category. The 'Headers' tab is selected, showing a table with one header entry: 'Authorization' with a checked checkbox, a value of 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bW...' and a description. Below this, there are empty rows for 'Key', 'Value', and 'Description'.

Key	Value	Description
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bW...	
Key	Value	Description



Para que serve esse teste?

Esse teste valida o fluxo completo de autorização da aplicação.

Ele confirma que:

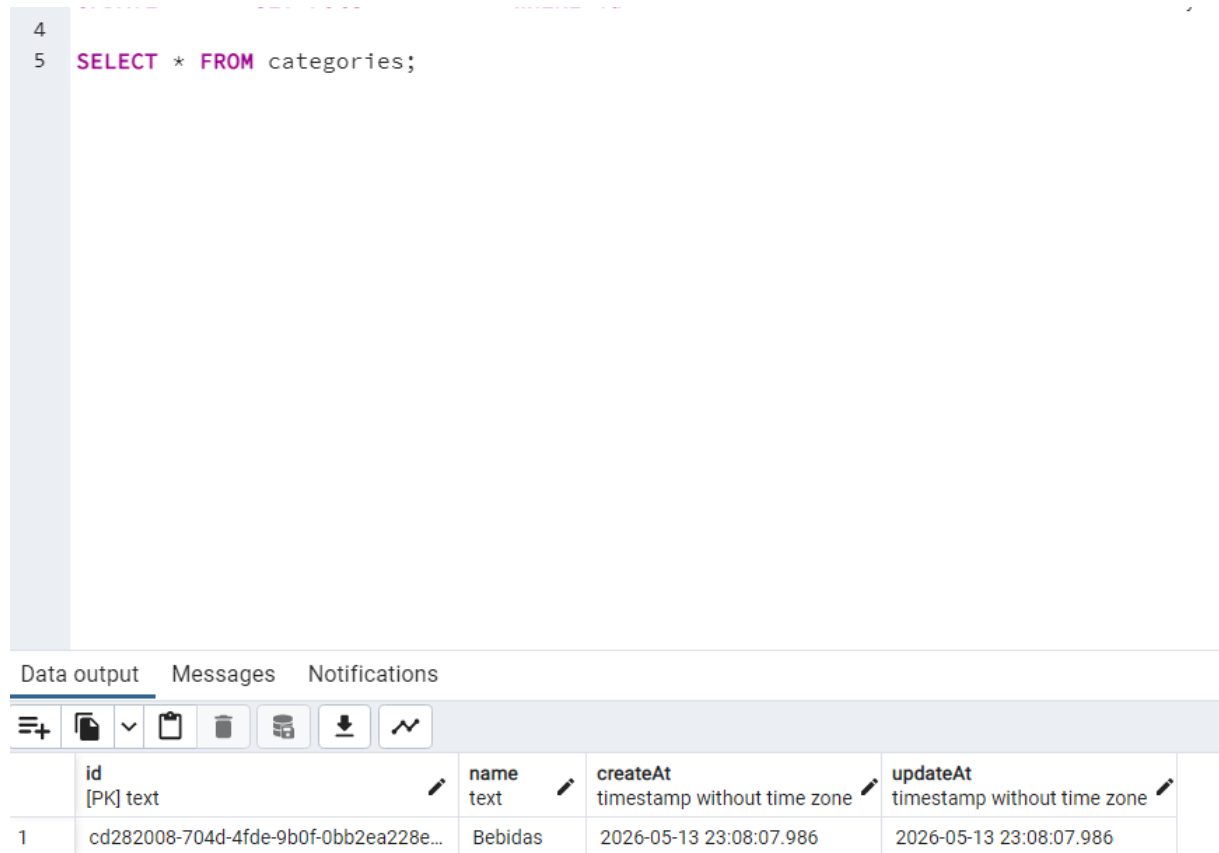
- o token JWT está sendo validado corretamente
- o middleware `isAuthenticated` está funcionando
- o middleware `isAdmin` está permitindo acesso apenas para administradores
- o cadastro da categoria está sendo salvo no banco de dados

Isso demonstra que a aplicação possui controle de acesso baseado em permissões funcionando corretamente.

Passo 9 – Verificar categoria cadastrada no banco

Agora abra o **pgAdmin** e execute o seguinte comando SQL:

```
SELECT * FROM categories;
```



The screenshot shows the pgAdmin interface. At the top, a SQL query is entered in the query editor: `SELECT * FROM categories;`. Below the editor, the 'Data output' tab is selected, displaying the results of the query in a table. The table has four columns: 'id' (text, primary key), 'name' (text), 'createAt' (timestamp without time zone), and 'updateAt' (timestamp without time zone). There is one row of data with the following values: id is 'cd282008-704d-4fde-9b0f-0bb2ea228e...', name is 'Bebidas', createdAt is '2026-05-13 23:08:07.986', and updateAt is '2026-05-13 23:08:07.986'.

	id [PK] text	name text	createAt timestamp without time zone	updateAt timestamp without time zone
1	cd282008-704d-4fde-9b0f-0bb2ea228e...	Bebidas	2026-05-13 23:08:07.986	2026-05-13 23:08:07.986

Para que serve esse teste?

Esse teste confirma que:

- a categoria foi realmente salva no banco de dados
- a rota de cadastro está funcionando corretamente
- o Prisma realizou o INSERT com sucesso
- a comunicação entre API e banco está operacional

Ou seja, valida que o fluxo completo de criação da categoria funcionou corretamente até a persistência no banco.

Passo 10 – Criar service de listagem de categorias

Agora, dentro da pasta **services/category**, crie um arquivo chamado **ListCategoryService.ts**.

Esse service será responsável por buscar e retornar todas as categorias cadastradas no banco de dados.

Adicione o seguinte código:

```
TS ListCategoryService.ts X
src > services > category > TS ListCategoryService.ts > ListCategoryService > execute > category
1  // Importa a instância do Prisma Client
2  // Responsável pela comunicação com o banco de dados
3  import prismaClient from "../../prisma";
4
5
6  // Classe responsável pela lógica de listagem de categorias
7  class ListCategoryService {
8
9      // Método assíncrono para buscar categorias no banco
10     async execute() {
11
12         // Bloco de tentativa para tratamento de erros
13         try {
14
15             // Busca todas as categorias cadastradas
16             const categories = await prismaClient.category.findMany({
17
18                 // Define quais campos serão retornados
19                 select: {
20                     id: true,
21                     name: true,
22                     createdAt: true,
23                 },
24
25                 // Ordena as categorias pela data de criação
26                 // "desc" = mais recentes primeiro
27                 orderBy: {
28                     createdAt: "desc",
29                 },
30             });
31
32             // Retorna a lista de categorias
33             return categories;
34
35             // Captura possíveis erros
36         } catch (erro) {
37

```

```
34
35     // Captura possíveis erros
36     } catch (erro) {
37
38         // Lança um erro personalizado
39         throw new Error("Falha ao buscar categorias");
40     }
41 }
42 }
43
44
45 // Exporta o service para ser utilizado no controller
46 export { ListCategoryService }
```

Para que serve esse código?

Esse service é responsável por listar todas as categorias cadastradas no sistema.

Ele:

- consulta a tabela categories no banco de dados
- retorna apenas os campos necessários (select)
- organiza os resultados pela data de criação
- trata possíveis erros durante a busca

Passo 11 — Criar controller de listagem de categorias

Agora, dentro da pasta **controllers/category**, crie um arquivo chamado **ListCategoryController.ts**.

Esse controller será responsável por receber a requisição de listagem de categorias e retornar os dados para o cliente.

Adicione o seguinte código:

```
TS ListCategoryController.ts 1 X
src > controllers > category > TS ListCategoryController.ts > ...
1  // Importa os tipos Request e Response do Express
2  // Request representa a requisição recebida
3  // Response representa a resposta enviada ao cliente
4  import { Request, Response } from "express";
5
6
7  // Importa o service responsável pela lógica de listagem de categorias
8  import { ListCategoryService } from "../../services/category/ListCategoryService";
9
10
11 // Classe responsável por controlar a listagem de categorias
12 class ListCategoryController {
13
14     // Método assíncrono responsável por processar a requisição
15     async handle(req: Request, res: Response) {
16
17         // Cria uma instância do service de listagem
18         const listCategory = new ListCategoryService();
19
20         // Executa o service para buscar as categorias no banco
21         const categories = await listCategory.execute();
22
23         // Retorna status 200 (OK) junto com a lista de categorias
24         res.status(200).json(categories);
25     }
26 }
27
28
29 // Exporta o controller para ser utilizado nas rotas
30 export { ListCategoryController };
```

Para que serve esse código?

Esse controller é responsável por controlar o fluxo de listagem das categorias.

Ele:

- recebe a requisição HTTP
- chama o service responsável pela consulta no banco
- retorna a lista de categorias para o cliente

Passo 12 — – Implementar rota de listagem de categoria

Agora, no arquivo **routes.ts**, adicione a rota responsável pela listagem de categorias.

Não se esqueça de importar o **ListCategoryController** no início do arquivo.

Adicione os códigos:

```
69 router.get("/category", // Endpoint da rota
70   | isAuthenticated, // Verifica se o usuário está autenticado
71   | new ListCategoryController().handle // Executa o controller responsável por listar as categoria
72 )
73
```

Para que serve esse código?

Essa rota permite listar todas as categorias cadastradas no sistema.

Ela:

- verifica se o usuário está autenticado
- executa o controller responsável pela listagem
- retorna as categorias cadastradas no banco de dados

Passo 13 – Testar listagem de categorias no Postman

Agora abra o **Postman** e realize uma requisição para testar a listagem das categorias cadastradas.

Configuração no Postman

- **Método:** GET
- **URL:** http://localhost:3333/category

Authorization

Essa rota é protegida, portanto é necessário enviar o token JWT.

Na aba **Authorization**:

- selecionar **Bearer Token**
- inserir o token do usuário autenticado

Ou via header:

Authorization: Bearer SEU_TOKEN_AQUI

Pizzaria Projeto / category / **Buscar categorias**

GET http://localhost:3333/category **Send**

Params Authorization **Headers (8)** Body Pre-request Script Tests Settings **Cookies**

Headers 7 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bm90...				
	Key	Value	Description			

Body Cookies Headers (8) Test Results **Status: 200 OK Time: 516 ms Size: 370 B Save as example**

Pretty Raw Preview Visualize JSON **Copy**

```
1 [
2   {
3     "id": "cd282008-704d-4fde-9b0f-0bb2ea228e74",
4     "name": "Bebidas",
5     "createdAt": "2026-05-13T23:08:07.986Z"
6   }
7 ]
```

Para que serve esse teste?

Esse teste confirma que:

- a rota de listagem está funcionando corretamente
- o middleware de autenticação está validando o token
- o service está buscando os dados no banco
- as categorias estão sendo retornadas corretamente pela API

Isso valida o fluxo completo de consulta das categorias cadastradas.